

P1481R0

constexpr structured bindings

Published Proposal, 2019-01-21

This version:

<https://wg21.link/pXXXX>

Author:

Nicolas Lesser (blitzrakete[at]gmail[dot][com])

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Analysis of allowing constexpr structured bindings

Table of Contents

1 Introduction

2 The Change

2.1 Function Arguments

2.2 Template Arguments

2.3 Lambdas and odr-use

2.4 `std::is_constant_evaluated`

2.5 Local variables

2.6 Data members

3 constexpr structured binding declarations

4 Conclusion

5 Proposed Wording

§ 1. Introduction

Since EWG encouraged further work for constexpr structured bindings with a (11-15-2-0-0) poll, this is a paper that explores the change needed to make them work.

§ 2. The Change

Currently, references are only allowed in a core constant expressions when they refer to an object which has a preceding initializer (function parameters for example don't) and its initializer has static storage duration (or if the reference is local to the expression).

```
void foo() {  
    const int a = 1;  
    static const int b = 1;  
  
    const int &refa = a;  
    const int &refb = b;  
    static_assert(refa == 1); // ill-formed, since 'a' has automatic storage  
    static_assert(refb == 1); // well-formed  
}
```

I propose to make references initialized by automatic storage duration objects be used as core constant expressions, which would make the `refa` line above well-formed. Since this is only a relaxation of the current rules, only code that was ill-formed previously will become well-formed. The only caveat is of course SFINAE, but I am not aware of any way to do it (since reference template arguments need to have static storage duration).

More formally, the definition of *usable in constant expressions* will change to allow references initialized with a core constant expression.

What follows is an enumeration of the possible impacted language features of the proposed change.

§ 2.1. Function Arguments

```
constexpr int foo(const int& p) {  
    static_assert(p == 1); // ill-formed  
    return p;  
}
```

No change. The `static_assert` is still ill-formed since `p` violates the "preceding initialization" rule.

§ 2.2. Template Arguments

```
template <const int& p> void foo();  
  
void bar() {  
    const int a = 0;  
    foo<a>(); // ill-formed  
}
```

No change. The call violates the requirement that `a` be a *constant-expression*.

§ 2.3. Lambdas and odr-use

```
void f() {  
    const int a = 1;  
    const int& b = a;  
  
    auto l1 = [] { return a; }; // well-formed  
    auto l2 = [] { return b; }; // ill-formed  
}
```

No change since `b` is a variable and for it not to be odr-usable it needs to be a *constant-expression*.

§ 2.4. `std::is_constant_evaluated`

```
void f() {  
    const int a = 1;  
    const int& b = std::is_constant_evaluated() ? a : 3;  
  
    int c[b]; // ill-formed - b == 3  
              // well-formed with proposed change - b == 1  
}
```

The above code will become well-formed as `b` will be a valid core constant expression.

§ 2.5. Local variables

```
void f() {  
    const int a = 2;  
    const int& b = a;  
    const int& c = b;  
  
    static_assert(b == 2); // ill-formed; well-formed with proposed change  
    static_assert(c == 2); // ill-formed; well-formed with proposed change  
}
```

This is the primary motivation for this change, since this will allow `constexpr` structured binding declarations.

§ 2.6. Data members

```
struct Foo {
    const int& foo;
};

void f() {
    const int a = 10;
    constexpr Foo f{a};
    static_assert(f.foo == 10); // ill-formed currently;
                                // well-formed with proposed change
}
```

`f.foo` will become a valid core constant expression.

§ 3. constexpr structured binding declarations

With the proposed change `constexpr` structured binding declarations will look like this:

```
void f() {
    constexpr auto[a] = std::tuple(1);
    // would be equivalent to
    constexpr auto __sb = Foo();
    const int& __a = std::get<0>(__sb);

    static_assert(a == 1); // ill-formed without the change, well-formed with
}
```

Note that the proposed change in this paper only applies to the tuple-like case of structured binding declarations. The two other cases do not need this change.

§ 4. Conclusion

In short, making references initialized with core constant expressions usable in constant expressions does not break anything in a major way while still allowing `constexpr` structured bindings. Any change of behavior is minimal, if any (SFINAE?). Only previously ill-formed code will become well-formed.

§ 5. Proposed Wording

Change [expr.const]p2 (7.7p2) as follows:

A variable is *usable in constant expressions* after its initializing declaration is encountered if it is a constexpr variable, or it is of reference type initialized with a core constant expression, or of const-qualified integral or enumeration type ~~, and its initializer~~ initialized with ~~is~~ a constant initializer.